

搜索的艺术

搜索与回溯 · 8题 · C++ & Python 双语对照

小鲁开始学习人工智能最基础的算法——搜索。"搜索就是穷举所有可能性——但穷举不等于蛮力。DFS回溯像走迷宫一条路走到黑，BFS像水的波纹层层扩散。加上剪枝——提前砍掉不可能的分支——搜索可以从指数级变成多项式级。理解搜索，是理解所有AI和优化算法的基石。"

DFS vs BFS 速查

- **DFS**: 深度优先——回溯探索。适合：全排列、N皇后、树遍历。空间O(深度)
- **BFS**: 广度优先——层层扩散。适合：最短路径、八数码、无权图最短路。空间O(宽度)
- **剪枝**: 提前排除不可能的分支——N皇后的col/dg/udg标记
- **状态编码**: 将棋盘/网格编码成字符串作哈希key——八数码的关键技术

NQ135 排列数字 AcWing 842

小鲁开始学习搜索算法："全排列——1到n的所有排列方式。用DFS深度优先搜索，像走迷宫一样一条路走到底再回头。每次选一个没用过的数放入路径，递归进入下一层，然后撤销选择(回溯)。"小华点头："DFS+回溯是搜索算法的灵魂框架。三个要素：路径(已经选了什么)、选择列表(还能选什么)、结束条件(选满了就输出)。这个框架将贯穿你的算法生涯。"

输入 一个整数n。

输出 1~n所有排列，每行一个，按字典序。

范围 $1 \leq n \leq 7$

输入	输出
3	1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1

思路 DFS回溯：path存当前路径，used标记已用数字。dfs(u): 若u==n则输出path；否则遍历1..n，若未used则标记used→加入path→dfs(u+1)→撤销标记→弹出path。复杂度O(n!)。注意递归树的深度=n，叶子数=n!。

技巧 Python: itertools.permutations一行。手写DFS重点理解回溯的'撤销'操作——进入递归前做的修改，返回后必须完全复原。回溯=尝试→失败→回退→再尝试。排列问题是回溯第一课，也是理解'搜索状态空间'的钥匙。

C++

```
#include <iostream>
using namespace std;

const int N = 10; //最大值
int n;             //读入的n
int path[N];       //记录路径每个位的值
bool used[N];      //记录第i个数是否被用过

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) cout << path[i] << " ";
        cout << endl;
        return;
    }
    //枚举数字1--n
    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            path[u] = i; //记录当前位path[u]的数字为i
            used[i] = true; //标记数字i为已经使用
            dfs(u + 1); //递归处理下一个位
            used[i] = false; //恢复现场
        }
    }
}

int main() {
    cin >> n;
    dfs(0);
    return 0;
}
```

Python

Python solution

NQ136 n-皇后问题 AcWing 843

"N皇后——在N×N棋盘上放N个皇后，使它们互不攻击（同行、同列、同对角线只有一个）。"小华画出棋盘，"回溯搜索的经典题。按行搜索——每行必放一个皇后，只需决定放在哪一列。用三个布尔数组标记：col[j]（列是否被占）、dg[i+j]（主对角线）、udg[i-j+n]（反对角线）。回溯的剪枝的力量——不满足约束的分支直接跳过。"

输入

一个整数n。

输出

所有方案，每个方案n行，每行n个字符(.Q)表示棋盘。

范围

 $1 \leq n \leq 9$

输入

4

输出

```
.Q..
...Q
Q...
..Q.

..Q.
Q...
...Q
.Q..
```

思路 按行DFS: dfs(r)——在第r行选一列放皇后。对每列, 检查col[j], dg[r+j], udg[r-j+n]是否未占。若可放则标记→递归r+1→撤销标记。对角线坐标映射: 主对角线r+j为定值([0,2n-2]), 反对角线r-j+n为定值。O(n!)但剪枝后实际更快。

技巧 对角线编号技巧: 主对角线r+c=常数(0到2n-2), 反对角线r-c+n=常数(n到2n+n)。用三个bool数组O(1)判断冲突——比每次遍历O(n)快得多。回溯+剪枝=搜索的效率保证——剪得越早, 省得越多。

C++

```
#include <iostream>
using namespace std;

const int N = 20; //最大值
int n; //输入的n
char g[N][N]; //棋盘
//三个指示器列、对角线、反对角线, 对角线以截距为编号
bool col[N], dg[N], udg[N];

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) puts(g[i]);
        puts("");
        return;
    }
    //枚举列0--n
    for (int i = 0; i < n; i++) {
        if (!col[i] && !dg[i + u] && !udg[i - u + n]) {
            g[u][i] = 'Q'; //当前位置设置为Q字符
            col[i] = dg[i + u] = udg[i - u + n] = true; //标记
            //数字i为已经使用郭
            dfs(u + 1); //递归
            //处理下一个位
            g[u][i] = '.'; //恢复
            //现场
            col[i] = dg[i + u] = udg[i - u + n] = false; //恢复
            //现场
        }
    }
}

int main() {
    cin >> n;
    //初始化图
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) g[i][j] = '.';
    dfs(0);
    return 0;
}
```

Python

Python solution

NQ137 走迷宫 AcWing 844

"从迷宫起点走到终点——每次只能上下左右走一步。"小华说, "DFS会找到一条路径, 但不一定最短; BFS广度优先——层层扩展, 像水的波纹——第一次遇到终点时走的一定是最短路径! BFS用队列: 起点入队→弹出一个位置→扩展四个方向→合法的入队。每个位置只访问一次。BFS是求无权图最短路径的标准算法。"

输入 第一行n,m。然后n×m的迷宫(0可走1墙)。

输出 从(0,0)到(n-1,m-1)的最少步数。

范围 $1 \leq n, m \leq 100$

输入

```
5 5
0 1 0 0 0
0 1 0 1 0
0 0 0 0 0
0 1 1 1 0
0 0 0 1 0
```

输出

8

思路 BFS模板：queue存(x,y,dist)；vis数组标记访问过。四个方向：dx=[-1,0,1,0],dy=[0,1,0,-1]。每次while queue非空：弹出队头，判断是否终点，否则扩展四个方向，合法且未访问则标记入队。每个位置最多访问一次，O(nm)。

技巧 BFS层序遍历——队列保证先访问近的点。dist不需要额外存储，直接存在队列元素或距离数组中。DFS和BFS的选择：DFS找一条路径(不保证最短)，BFS找最短路径。Python用deque作BFS队列——popleft() O(1)。

C++

```

#include <algorithm>
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

typedef pair<int, int> PII; // Y总风格pair<int,int>声明

const int N = 107;

int n, m;
int g[N][N], d[N][N];

//广搜
int bfs() {

    queue<PII> q; //队列

    memset(d, -1, sizeof d);
    d[0][0] = 0;
    q.push({0, 0});

    // 四个方向向量
    int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

    while (q.size())
    { //队列为空的时候退出广搜
        auto t = q.front(); //取队头
        q.pop(); //弹出队头

        //搜索四个方向
        for (int i = 0; i < 4; i++)
        {
            int x = t.first + dx[i], y = t.second + dy[i]; //
            待搜索的新坐标点

            if (x < 0 || x >= n || y < 0 || y >= m || g[x][y] !
            = 0 || d[x][y] != -1) continue;

            d[x][y] = d[t.first][t.second] + 1;

            q.push({x, y}); //入队

        }
    }

    return d[n-1][m-1];
}

int main() {

    cin >> n >> m;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> g[i][j];

    cout << bfs() << endl;

    return 0;
}

```

Python

Python solution

NQ138 八数码 AcWing 845

"3×3的格子，8个数+1个空格——通过滑动空格，把乱序排列恢复成123/456/78x。"小华说，"BFS的状态空间搜索！每个状态是一个3×3排列→看成一个字符串。从初始状态开始BFS，每次空格和相邻数字交换生成新状态。用字典记录每个状态到初始状态的距离。终止状态固定为'12345678x'。BFS第一次遇到终止状态时的距离就是最少步数。"

输入 一行9个字符，表示初始状态（x代表空格）。

输出 最少移动步数，无解输出-1。

范围 3×3网格

输入

2 3 4 1 5 x 7 6 8

输出

19

思路 BFS状态空间搜索：状态编码为字符串(9字符)。从初始状态开始BFS，用unordered_map/dict记录距离。每次找到'x'的位置，和上下左右(若合法)交换，生成新状态。复杂度 $O(9!)=362880$ 状态——BFS完全可以。注意逆序对奇偶性判断无解。

技巧 BFS的队列保证层层扩展——第一次遇到目标状态时步数最少。状态总数 $=9!/2=181440$ （一半无解）。Python用字典作距离映射+deque作BFS队列。把3×3网格编码成一维字符串是简化状态的关键。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <unordered_map>
#include <queue>

using namespace std;

int bfs(string start)
{
    string end = "12345678x";

    queue<string> q;
    unordered_map<string,int> d;//字符串映射到整数

    q.push(start);
    d[start] = 0;//map的用法

    //方向向量
    int dx[4] = {1,-1,0,0},dy[4]={0,0,1,-1};

    //广搜
    while(q.size())
    {
        auto t = q.front();
        q.pop();

        int distance = d[t];
        if (t==end) return distance;//找到目标

        //查询x在字符串中的下标
        int k = t.find('x');
        int x = k / 3, y = k % 3;//转换为宫格坐标

        for (int i = 0; i < 4; i ++ )
        {
            int a = x + dx[i], b = y + dy[i];

            if (a >= 0 && a < 3 && b >= 0 && b < 3)
            {
                swap(t[k], t[a*3+b]);//移动x的位置
                if (!d.count(t))
                {
                    d[t] = distance + 1;
                    q.push(t);//压入队列
                }
                swap(t[k],t[a*3+b]);//还原现场
            }
        }
    }
    return -1;
}

int main()
{
    string c, start;
    //去掉空格
    for (int i = 0; i < 9; i ++ )
    {
        cin>>c;
        start += c;
    }
}

```

Python

Python solution

```
cout<<bfs(start)<<endl;  
}
```

NQ139 树的重心 AcWing 846

"找树的重心——删除重心后剩余各连通块中最大块的最小值。"小华画出树，"DFS遍历树，统计每个节点的子树大小。删除节点u后，连通块有三部分：u的各子树、u的父节点所在的整棵树减去u的子树。取这些中的最大值，然后所有u的这个最大值中的最小值就是答案。树形DP+DFS——理解'树遍历+状态统计'的模式。"

输入 第一行n。然后n-1行每行a b表示边。

输出 重心对应的最大连通块最小值。

范围 $1 \leq n \leq 10^5$

输入	输出
9	4
1 2	
1 7	
1 4	
2 8	
2 5	
4 3	
3 9	
4 6	

思路 DFS统计子树大小：dfs(u,fa)——遍历u的邻居v(排除fa)，dfs(v,u)， $s[u] += s[v]$ 累加子树大小。删除u后最大块= $\max(s[v](\text{各子树}), n-s[u](\text{上方树}))$ 。取所有u的max的最小值。 $O(n)$ 一次DFS。

技巧 树形DFS的关键：用fa参数避免回头，用邻接表存图。Python递归可能超深度——用sys.setrecursionlimit(200000)或手写栈。重心在点分治(CDQ)中起关键作用——保证递归深度为 $\log n$ 。这是树形DP的入门。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = N*2; // 无向图

int n;
int h[N], e[M], ne[M], idx;
int ans = N;
bool st[N]; // 顶点n是否访问

void add(int a, int b) // 添加一条边a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int dfs(int u)
{
    st[u] = true;

    int size = 0; // 去掉i节点后, i子树的最大子树节点数
    int sum = 0; // i子树的节点总数
    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (st[j]) continue;

        int s = dfs(j); // 返回j子树的大小
        size = max(size, s);
        sum += s; // 计算所有子树和
    }

    size = max(size, n - sum - 1); // i子树和与 非i子树部分的和, 取最大值
    ans = min(ans, size);

    return sum+1; // 加上i节点本身
}

int main()
{
    cin >> n;
    memset(h, -1, sizeof h);
    for (int i = 0; i < n-1; i ++ )
    {
        int a, b;
        cin >> a >> b;
        add(a, b), add(b, a); // 无向边
    }
    dfs(1);
    cout << ans << endl;
    return 0;
}

```

Python

Python solution

"给一个有向图，求从1号点到n号点的最短距离（每条边长度为1）。"BFS天然适合！因为边权相同，BFS第一层是距离1的点，第二层距离2.....第一次遇到n号点时的距离就是最短距离。小鲁说："这和迷宫BFS本质上是一样的——只是图的连接关系用邻接表存储，而不是四方向规则。"

输入 第一行n,m。然后m行每行a b表示有向边。

输出 1到n的最短距离，不可达输出-1。

范围 $1 \leq n, m \leq 10^5$

输入	输出
4 5	1
1 2	
2 3	
3 4	
1 3	
1 4	

思路 BFS模板：queue存(node, dist)，vis数组标记访问。邻接表存图：vector adj[N]或list of lists。从1号出发BFS，遇到n时输出dist。每个节点最多访问一次， $O(n+m)$ 。BFS是边权相同的单源最短路径标准算法。

技巧 邻接表：Python用列表嵌套——adj=[[] for _ in range(n+1)]。BFS的queue用deque。dist数组初始化为-1表示未访问，dist[1]=0。BFS层层扩展的性质保证第一次遇到终点时的距离就是最短的。无权图的最短路=BFS。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>

using namespace std;

const int N = 100007; // 有向图

int n, m;
int h[N], e[N], ne[N], idx;
int d[N];

void add(int a, int b) // 添加一条边 a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int bfs()
{
    memset(d, -1, sizeof d);
    queue<int> q;
    d[1] = 0; // 起始点
    q.push(1);

    while (q.size())
    {
        auto t = q.front();
        q.pop();

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (d[j] == -1)
            {
                d[j] = d[t] + 1; // 层次加1
                q.push(j);
            }
        }
    }
    return d[n];
}

int main()
{
    cin >> n >> m;
    memset(h, -1, sizeof h);

    for (int i = 0; i < m; i++)
    {
        int a, b;
        cin >> a >> b;
        add(a, b);
    }

    cout << bfs() << endl;
    return 0;
}

```

Python

Python solution

"求一个字符串的最小循环节——即最小的k使得s等于某个字符串重复k次。"小华说，"方法1：枚举子串长度len（必须是n的约数），检查所有对应位置字符是否相同。方法2：KMP的next数组——最小循环节= $n - \text{next}[n]$ （当 $n \% (n - \text{next}[n]) == 0$ 时）。前者 $O(n \times \text{约数个数})$ ，后者 $O(n)$ 。两种方法从不同角度展示了'周期性判断'的思维。"

输入 多行，每行一个字符串，以'.'结束。

输出 每行输出最大k值。

范围 字符串长度 $\leq 10^6$

输入	输出
abcd	1
aaaa	4
ababab	3
.	

思路 KMP法：next[n]是n的最长公共前后缀长度。若 $n \% (n - \text{next}[n]) == 0$ ，则最小循环节 $\text{len} = n - \text{next}[n]$ ， $k = n / \text{len}$ 。否则整个串是最大循环节($k=1$)。验证：aaaa中 $\text{next}[4]=3$ ， $n - \text{next}[n]=1$ ， $k=4$ 。枚举法：检查len为n的约数，验证所有 $s[i] == s[i \% \text{len}]$ 。

技巧 KMP的next数组不仅用于匹配，还揭示了字符串的周期结构—— $\text{len} = n - \text{next}[n]$ 是最小可能的循环节长度。这个性质在字符串周期分析和压缩中非常有用。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string str;

    while (cin >> str, str != ".")
    {
        int len = str.size();

        for (int n = len; n; n -- )
            if (len % n == 0)
            {
                int m = len / n;
                string s = str.substr(0, m);
                string r;
                for (int i = 0; i < n; i ++ ) r += s;

                if (r == str)
                {
                    cout << n << endl;
                    break;
                }
            }

        return 0;
    }
}
```

Python

Python solution

NQ142 字符串最大跨距 AcWing 778

"找字符串S中三个子串S1,S2,S3的位置——S1在最左，S2在最右，S3在S1和S2之间——求S1最右位置到S2最左位置的距离。"小鲁说："用find()从左往右找S1的最左出现，用rfind()从右往左找S2的最右出现。然后检查S3是否在它们之间合法存在。Python的str.find/rfind让这题行云流水。"

输入 一行字符串S，以及S1,S2,S3（用逗号隔开）。

输出 最右跨距，若无合法方案输出-1。

范围 S长度 ≤ 300

输入	输出
abcd123ab888efghij45ef67kl,ab,ef,45	18

思路 1)S.find(S1)→左边界L。2)S.rfind(S2)→右边界R。3)检查S[L+len1:R]中是否有S3且位置合法。4)R-(L+len1)就是跨距。若L==-1或R==-1或R

技巧 Python的find()返回第一次出现，rfind()返回最后一次出现。这是字符串操作的经典组合——左找+右找+中间检查。逗号分割输入：s=input().split(',')。本题训练的是对字符串多种查找操作的组合运用能力。

C++

```

#include <iostream>

using namespace std;

int main()
{
    string s, s1, s2;

    char c;
    while (cin >> c, c != ',') s += c;
    while (cin >> c, c != ',') s1 += c;
    while (cin >> c) s2 += c;

    if (s.size() < s1.size() || s.size() < s2.size()) put
s("-1");
    else
    {
        int l = 0;
        while (l + s1.size() <= s.size())
        {
            int k = 0;
            while (k < s1.size())
            {
                if (s[l + k] != s1[k]) break;
                k ++ ;
            }
            if (k == s1.size()) break;
            l ++ ;
        }

        int r = s.size() - s2.size();
        while (r >= 0)
        {
            int k = 0;
            while (k < s2.size())
            {
                if (s[r + k] != s2[k]) break;
                k ++ ;
            }
            if (k == s2.size()) break;
            r -- ;
        }

        l += s1.size() - 1;

        if (l >= r) puts("-1");
        else printf("%d\n", r - l - 1);
    }

    return 0;
}

```

Python

Python solution

本章知识点总结

- DFS回溯：路径+选择列表+结束条件。回溯撤销操作是理解搜索的关键
- BFS：队列+距离数组。层层扩展保证最短路径（边权相同）
- 状态搜索：八数码——将棋盘编码为字符串做状态BFS

- 树形DFS：重心问题——理解树遍历+子树统计的DP模式

— 第13章完 · 共8题 · 自强不息 —